

AI-Signal-Pipeline: Architecture & Production Design

Data Intelligence Pipeline for the AI/Venture Ecosystem Graph

1. Scale Strategy: Collecting 500,000+ Records Without Manual Intervention

The current implementation (papers, startups, products) is built async-first using aiohttp with bounded concurrency, and deliberately favors structured APIs / static feeds (YC's public companies feed, Product Hunt's GraphQL API, HuggingFace's Daily Papers API) over raw HTML scraping wherever a legitimate one exists. This removes anti-bot risk entirely for those verticals and is the primary reason the 1,000-record minimums were hit reliably. Scaling to 500k+ requires no code changes -- only infrastructure scaling -- via the following pattern:

- **Horizontal worker pool:** each scraper module (startups, products, papers, news, jobs) runs as an independent async worker process. A lightweight task queue (Redis + RQ, or Celery) distributes source-URLs/pages across N worker replicas, so scaling from 1,000 to 500,000 records is a matter of increasing replica count, not rewriting logic.
- **Per-domain rate limiting:** a token-bucket limiter keyed by domain (not global) ensures workers scale independently per source without one slow/strict site throttling the whole pipeline.
- **Playwright worker pool for JS-heavy/Cloudflare-protected sources:** a small, separately-scaled pool of headless-browser workers (Playwright Async, stealth-patched) handles the minority of sources that lack an API and are anti-bot protected, since these are far more resource-intensive per request than the aiohttp-based API workers.
- **Idempotent, resumable workers:** each worker writes progress checkpoints (last-processed cursor/page) to shared storage, so a crashed or restarted worker resumes rather than re-scraping from zero -- essential once a single run spans hours across hundreds of thousands of records.

2. Handling 413s and 429s Across Thousands of Concurrent Extractions

Implemented in `llm_orchestrator.py` and proven working end-to-end against live Gemini and Groq endpoints during this trial:

- **413 (payload too large):** a paragraph-and-sentence-aware chunker splits raw text into sub-12,000-character segments before the first LLM call, preserving semantic boundaries rather than blind character truncation. If a provider still returns 413 (a stricter per-model limit), the offending chunk is recursively re-split at half size and retried -- this is self-correcting rather than relying on a single fixed chunk size guess.
- **429 (rate limited):** exponential backoff with jitter ($\text{base delay} \times 2^{\text{attempt}} + \text{random } 0\text{-}1\text{s}$) is applied per tier, up to 3 retries, before falling through to the next tier in the fallback chain (Gemini Flash to Groq Llama/OSS to DeepSeek). This was validated live: when the configured Gemini model returned a transient error, the chain correctly fell through and Groq completed the extraction with a valid structured JSON result.
- **At scale (thousands of concurrent extractions):** a semaphore-bounded worker pool caps in-flight requests per provider to stay under published rate limits proactively, rather than relying solely on reactive backoff -- reducing wasted 429 round-trips as concurrency grows.

3. Freshness Tracking Across Distributed Crawler Nodes

The news and jobs crawlers enforce a strict 24-hour publish-date freshness window (verified against real RSS/API timestamps, not scrape-time), deliberately discarding anything that cannot be confidently dated rather than guessing. To extend this to distributed, never-reprocess-twice operation at scale:

- **Shared dedup store:** a Redis set (or Postgres unique constraint) keyed on a normalized content fingerprint (source domain + canonical URL, or a content hash for sources with unstable URLs) is checked before writing any record. Distributed worker nodes share this store, so the same article/job is never double-counted regardless of which node processed it.
- **Per-source cursor persistence:** each source stores its last-successfully-processed timestamp/ID. On the next scheduled run, workers only request items newer than that cursor -- this is how the freshness guarantee holds even across scheduled reruns (e.g., hourly cron), rather than re-scanning the entire feed each time.

- **Scheduled, not one-shot, execution:** a production deployment runs the news/jobs crawlers on a recurring schedule (hourly) via a lightweight orchestrator (cron, Airflow, or a simple asyncio scheduler), since posting frequency varies by source -- a single point-in-time run can legitimately see zero fresh items from a lower-volume source, which is correct behavior, not a bug.

4. Storage Strategy: Primary DB and Vector/Graph Storage

Layer	Choice	Justification
Primary store	PostgreSQL	ACID guarantees for the canonical entity records (startups, products, papers, jobs, news); native JSONB columns accommodate the schema's nested content.* fields without a rigid migration for every new field; mature tooling for the dedup/freshness cursor tables described above.
Relationship / graph layer	Neo4j (introduced at scale)	The brief's core value is the *graph* -- startup to product to paper to GitHub repo. At current trial scale, foreign keys in Postgres suffice; at 500k+ records with multi-hop queries (e.g., "papers whose authors later founded a startup that launched a product"), a native graph DB avoids expensive recursive joins.
Vector store	Pinecone or Chroma	Embeddings of paper abstracts / product descriptions enable semantic search and duplicate/near-duplicate detection for entity resolution beyond what string-similarity (rapidfuzz) alone catches -- e.g., catching a renamed or rebranded startup that a fuzzy string match would miss.

Current trial implementation: Google Sheets serves as the deliverable output format per the brief's submission requirement. In production, Sheets would be replaced by the Postgres/Neo4j/vector-store combination above, with Sheets (or a BI dashboard) generated as a periodic export view for stakeholders rather than the system of record.